

THE AI TOOLBOX · CLASS 10

Linear Regression

Build your first predictive model — and see what "**learning**" means numerically.

Merit Point Academy · 60 min · press **S** notes · → next ·  to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

Charts argue

Every chart answers a one-sentence question — and always plot your data (Anscombe!).

Correlation $\neq$ causation

r measures straight-line strength; it never proves what drives what.

Honest axes

Truncated axes lie; self-contained captions don't.

Homework share: your 3-chart data story's one-sentence conclusion. Notice something? Your scatter plots already *showed* a relationship between two variables — **today you turn that relationship into a machine that predicts.** The data stage is done; the model stage begins.

Where we're going

- **0–8'** Homework share + why we call it "regression"
- **8–18'** Features → target · loss (MSE) · the idea in one picture
- **18–30'** Gradient descent: how a model actually learns
- **30–38'** Train vs. test — the honest evaluation, again
- **38–46'** Demos: fit a regressor in Colab · the AI coefficient-reader
- **46–54'** Two labs: gradient descent from scratch · the AV distance model
- **54–60'** Your prediction model + error plot, homework & wrap-up
- Quizzes &  comments throughout

By the end: explain what a regression model does, understand loss and how a model "learns," and train + honestly evaluate a predictor.

WARM-UP · ACTIVATE

Quick check · tap an answer

A taller bounding box in a camera frame usually means a closer object. Could a single straight line turn box height into a distance estimate?

No — distance is far too complex for a straight line

Yes, in principle: fit a line through (box height, distance) examples, then read predictions off it

Only if you already know the camera's focal length by hand

Lines can only describe things that are already exactly straight, like roads

Why we call it "regression"

In the 1880s, **Francis Galton** measured the heights of hundreds of parents and their grown children. He noticed that very tall parents tended to have tall children — but **slightly closer to average**. Heights "regressed toward the mean." The line he drew through that cloud of points gave us the name for the whole method we still use today: **regression**.



Speak this story

Neural voice · click to play / pause



The METHOD outlived the specific finding — "regression" stuck as the name for "fit the best line through a cloud of points," whatever the cloud is about. Today's cloud: box height and distance.

CONCEPT 1 · THE SHAPE OF THE PROBLEM

Features → target



Features (X)

What you **already know** — box height in pixels, hours studied, square footage. Class 8 called these "columns"; here they're the model's inputs.



Target (y)

What you want to **predict** — distance in meters, a test score, a price. A number, not a category (that's Class 11).

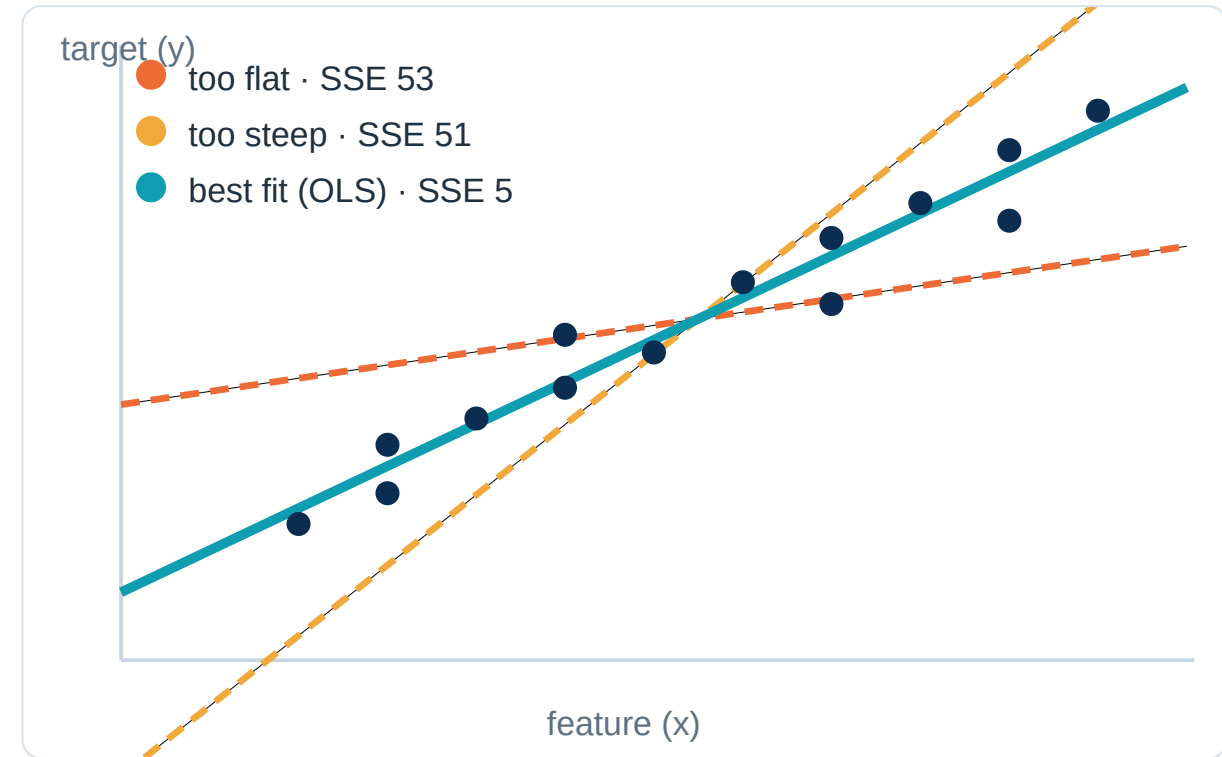
KEY IDEA — a model is just a **learned relationship** between features and a target. Regression finds the line (or hyperplane, with more features) that describes that relationship best. "Best" gets a precise meaning next.

Loss: how wrong are we?

"Best fit" means **smallest error**. Loss (here, **Mean Squared Error**) measures total wrongness — training nudges the line to make loss as small as possible.

$$\text{MSE} = \text{mean}((y_{\text{true}} - y_{\text{pred}})^2)$$

Squaring does two jobs: it kills the +/- sign problem, and it punishes big misses harder than small ones.



Look at the numbers: the too-flat and too-steep lines have huge SSE (~52-53); the OLS best-fit line's SSE is a fraction of that (~5). **Lower loss isn't always better** — overfitting (Class 3!) is what happens when a model chases loss all the way to zero by memorizing noise instead of learning the trend.

Quick check · tap an answer

Two candidate lines fit the same cloud of points. Line A has $SSE = 12$. Line B has $SSE = 340$. Which line does the data prefer, and why?

Line B — bigger numbers mean a stronger model

Line A — its predictions are collectively much closer to the true values

Neither — SSE doesn't measure fit quality

They're equally good; SSE only matters for classification

Gradient descent: walking downhill blindfolded

Picture the loss for every possible (m, b) as a **landscape of hills and valleys**. Training starts somewhere on that landscape and takes small steps **downhill** — toward lower loss — until it reaches the valley floor.

- At your current spot, compute the **gradient**: which direction is steepest *uphill*
- Step the **opposite** way — downhill — by a small amount
- Repeat, thousands of times, until the loss stops dropping



Mountain-descent analogy:

eyes closed, feel which way is downhill, take a step, repeat — you'll reach the bottom without ever seeing the whole mountain.

The step size is the *learning rate*. Too small: painfully slow. Too large: you overshoot the valley and bounce around — or never converge at all. Lab 1 today: you implement this exact loop yourself, from scratch.

Train on some, judge on the rest



The examples the model **sees and learns from** — gradient descent adjusts m and b against this pile only.



Examples the model has **never seen**. Scoring here is the only honest measure of whether it actually learned the pattern — or just memorized the training rows.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
```

Recognize this? It's Class 3's **three-pile discipline**, back for its first real workout: train to fit, test to judge. Score on training data and you're measuring memorization, not learning — the same trap the overfitting quiz warned about two classes ago.

Distance from box height

A real monocular-depth heuristic, built as regression: **bigger box** → **closer object**, roughly linearly.

1

Feature

Bounding-box height, in pixels

2

Target

Distance to the object, in meters

3

Check

Plot predicted vs. actual — where does the line break down?

Spoiler for step 3: within the training range, the fit is strong. But push a box height *far* beyond what the model ever saw — a very close object — and the line can predict a **negative distance**. A car can't be −6 meters away. Lab 2 makes you watch this happen. Prefer another domain? Same three steps, any relationship.

DISCUSS · 5 MINUTES · SPOT THE LINE

What rises together in a straight line?

Name two things in the real world you think move together in a roughly **straight-line** relationship.

Propose one

Hours studied vs. test score? Height vs. shoe size? Screen time vs. sleep?
Pick something you could actually measure.

Stress-test it

Would a straight line *really* fit, or does it curve at the extremes? What would break the relationship entirely?

Pairs, 3 minutes — best pair + best "where it breaks" in .

LIVE DEMO 1

Fit a regressor in Colab, five lines

The whole workflow, live: split, fit, predict, score — then plot predicted vs. actual so the fit is visible, not just a number.

```
You ▶ X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,  
random_state=42)
```

```
You ▶ model = LinearRegression(); model.fit(X_train, y_train)
```

```
You ▶ pred = model.predict(X_test)
```

```
You ▶ print(mean_squared_error(y_test, pred), r2_score(y_test, pred))
```

```
You ▶ plt.scatter(y_test, pred); plt.xlabel("actual"); plt.ylabel("predicted");  
plt.title("...")
```



colab.research.google.com

LIVE DEMO 2

The AI coefficient-reader

Class 4's mentor hat, aimed at a trained model: turn raw numbers (coef_, intercept_) into a sentence a human understands.

You ▶ My model: $\text{distance} = -0.32 \times \text{box_height} + 44.4$. Explain what the coefficient means in plain English.

You ▶ R^2 on my test set is 0.97. Is that good? What could make it misleading?

You ▶ If I feed this model a box_height way outside my training data's range, should I trust the prediction? Why or why not?

You ▶ Suggest one more feature that might improve this distance estimate.



chatgpt.com

Gradient descent, from scratch — watch it find the line

No scikit-learn yet — just the loop itself. Watch loss fall as two numbers (m, b) walk downhill toward the same answer a calculator would give you.

```
1 # Study hours -> test score. Let's find the best line OURSELVES.
2 hours = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3 scores = [52, 55, 59, 65, 68, 74, 77, 83, 87, 91]
4 n = len(hours)
5
6 m, b = 0.0, 0.0      # start anywhere - even (0,0) is fine
7 lr = 0.01           # the learning rate: how big each downhill step is
8
9 for step in range(2000):
10     preds = [m*x + b for x in hours]
11     errors = [p - y for p, y in zip(preds, scores)]
12     loss = sum(e**2 for e in errors) / n      # MSE
```



Run



Explain each line



Explain



Debug



Improve



Mira

Python loads on first Run

► Run the code to see the output here.

Fit the distance model — then push it too far

Real scikit-learn, real train/test split, an honest test score — and then the extrapolation experiment the case study spoiled. Watch a good model say something impossible.

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import mean_squared_error, r2_score
4
5 # Bounding-box height (px) -> distance (m). Real detection-log shape:
6 # bigger box = closer object, roughly linearly, with real-world noise.
7 box_h = [19.7, 22.9, 27.5, 27.3, 30.0, 33.0, 33.1, 37.6, 40.8, 44.3,
8          42.6, 46.3, 47.5, 54.4, 56.2, 54.8, 62.9, 65.3, 65.9, 68.2,
9          67.9, 69.6, 75.2, 74.9, 78.1, 81.0, 82.2, 87.3, 89.6, 94.6,
10         95.1, 98.3, 100.0, 103.5, 104.7, 106.2, 113.0, 115.5, 117.0, 118.7]
11 dist    = [37.8, 36.3, 35.1, 34.1, 36.7, 33.9, 36.1, 32.4, 34.2, 32.6,
12           28.9, 28.7, 31.9, 27.4, 29.4, 27.0, 22.7, 24.8, 25.3, 22.0]
```



Run



Explain each line



Explain



Debug



Improve



Mira

Python loads on first Run

► Run the code to see the output here.

YOUR PROJECT

Build a prediction model

Train a regression model on real data and see how close its predictions actually land — **judged honestly, on data it never saw.**

- Load a dataset — your own Class-7/8 data, or box-height → distance
- Split into train and test **before** touching the model
- Fit a LinearRegression
- Plot **predicted vs. actual**, labeled

Deliverable

A working regressor plus an error plot showing its accuracy.

Success looks like

Scored on a held-out test set — and the predicted-vs-actual plot is labeled.

BUILD IT NOW · HANDS-ON

Fit, split honestly, plot the truth

1. Pick your feature(s) and target — a numeric column you want to predict
2. `train_test_split` **before** anything else touches the model
3. `model.fit()` on train, `model.predict()` on test — never the reverse
4. Report MSE and R^2 on the **test** set only
5. Plot predicted vs. actual — labeled axes, a title, a diagonal reference line if you can
6. One sentence: where does your model do well, and where might it break down?

Colab

scikit-learn

matplotlib

ChatGPT / Claude

Instructor's rule: always score on test data — training accuracy will lie to you, cheerfully and confidently, exactly like the model in Lab 2's in-range numbers looked great until you pushed past the edge.

BENCHMARK · JUDGE AGAINST THIS

What a finished prediction model looks like



The fit

$\text{distance} = -0.32 \times \text{box_h} + 44.4$

One coefficient, one intercept — the whole learned relationship, readable in a sentence.



The honest score

Test $R^2 = 0.97$, test MSE = 2.2 — computed on the 25% the model never trained on. Not the training score.



The plot

Predicted vs. actual, title + labeled axes, dots hugging the diagonal — a stranger can see the fit is good *without you narrating it* (Class 9's rule).

The bar: a coefficient you can explain in one sentence, a score computed honestly on unseen data, a labeled chart that proves it — and, bonus, one sentence about where the model would break (Lab 2's extrapolation lesson, applied to your own data).

Today's vocabulary

Feature / target — what you know (input) vs. what you predict (output). X and y .

Gradient descent — repeatedly stepping downhill on the loss landscape until it stops falling.

R^2 (R-squared) — the fraction of the target's variation the model explains; 1.0 is perfect, 0 is no better than guessing the average.

Loss (MSE) — mean squared error: how wrong the model's predictions are, on average.

Learning rate — the size of each downhill step; too big overshoots, too small crawls.

Extrapolation — predicting outside the range of training data; a line has no edges, so it will answer anyway — often wrongly.

RECOMMENDED HOMEWORK

Before next class

1. **Finish your prediction model** (fit + honest test score + labeled predicted-vs-actual plot) — submit the Colab link in Google Classroom.
2. **Find your model's edge:** push one feature value well outside your training range (like Lab 2's `box_height=160`) and report what the model predicts. Is it plausible?
3. **One-sentence gradient descent recap:** in your own words, what does "training" a linear model actually DO, step by step?

★ Stretch (optional)

Kaggle's "**House Prices — Advanced Regression Techniques**": fit a linear regression using just ONE feature (like square footage), then plot the scatter + your fitted line. How much does one number explain?

[kaggle.com · House Prices](https://www.kaggle.com/c/house-prices-advanced-regression-techniques)

Task 2 is tonight's most important habit: every model you ever ship should know — and you should know — where its training data stops. Questions → 💬 on any slide.

TODAY'S LINE TO REMEMBER

"All models are wrong, but some are useful."

— George E. P. Box, statistician

Your line isn't reality — it's a simplification that works **within its range**, and breaks honestly outside it.



Speak this

WRAP-UP

Three things to keep

- A model is a **learned relationship**: features in, a target predicted out
- **Loss measures wrongness; gradient descent walks downhill** until it stops falling
- Judge on **test data only** — and never trust a prediction far outside the training range

Exit ticket: name one feature that would improve your distance predictor. Post it in .

Next · Class 11 — predict categories, not numbers: classification and logistic regression.

BACKUP MATERIAL

Extra slides — if there's time

Deeper dives and extra practice. Skip in a tight hour; pull up on demand or for a longer session.

Multiple regression

more than one feature

Regularization, previewed

penalizing complexity

Bonus quiz

one more check

Multiple regression: more inputs, same idea

One feature

$$\text{distance} = m \cdot \text{box_height} + b$$

A line through a 2D cloud (height vs. distance).

Many features

$$\begin{aligned} \text{distance} = & m_1 \cdot \text{box_height} \\ & + m_2 \cdot \text{box_width} + b \end{aligned}$$

A flat plane (or hyperplane) through a higher-dimensional cloud — one weight per feature.

The code barely changes — x just gets more columns. Each coefficient answers "holding everything else fixed, how much does THIS feature move the prediction?" — exactly what demo 2's AI coefficient-reader was interpreting.

What if the model tries too hard?

The problem

With many features, plain regression can assign huge weights to noisy columns — memorizing quirks instead of the real trend. Sound familiar? It's **overfitting** (Class 3), wearing a regression costume.

The idea (Ridge / Lasso)

Add a penalty for **large coefficients** to the loss. The model must now justify every big weight — smaller, steadier coefficients that generalize better to the test set.

You don't need the math today — just the shape of the idea: **penalize complexity, not just error**. This is the family of fixes that answers Lab 2's Challenge 2 ("what would you add before shipping this?").

Quick check · tap an answer

Your model gets $R^2 = 0.99$ on the TRAINING set but $R^2 = 0.41$ on the TEST set. What's the honest headline?

The model is excellent — report the 0.99

The model is overfitting: it memorized training quirks and generalizes poorly — the 0.41 is the truth

Test sets are unreliable; trust training scores

R^2 can't be compared between train and test